

APPENDICES

I. BILL OF MATERIALS

The technology and software needed to design and implement the Blockchain-Based Ayurvedic Product Traceability System are listed in the Bill of Materials. Ensuring product authenticity, transparency, and safe tracking throughout the supply chain are the main goals of this system. The following is a list of the parts used in this project:

1. The system is primarily software-based, which lowers the cost of hardware and makes implementation and scalability simple.
2. For development and operation, a computer or laptop with a regular processor, 4–8 GB of RAM, and a few basic peripherals is needed.
3. Blockchain access, testing, deployment, and real-time data updates all depend on a reliable internet connection.
4. While Node.js or Python are used in the backend for system functionality, HTML, CSS, and JavaScript are used in the frontend.
5. For safe tracking, automation, and product authentication, smart contracts, blockchain platforms (Ethereum/Hyperledger), and QR code technologies are utilised.
6. Product-related data is effectively stored and managed using database systems like MySQL or MongoDB.
7. Coding, version control, deployment, and scalability are all accomplished via development tools like Visual Studio Code, Git, and cloud platforms.

II. CODING

```
from flask import Flask
from flask_cors import CORS
from config import Config
from models import db
from api_routes import api
```

```

from web_routes import web
import os
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

def create_app(config_class=Config):
    """Application factory pattern"""
    app = Flask(__name__)
    app.config.from_object(config_class)

    # Initialize extensions
    db.init_app(app)
    CORS(app)

    # Register blueprints
    app.register_blueprint(api)
    app.register_blueprint(web)

    # Create database tables
    with app.app_context():
        db.create_all()
        init_sample_data()

    return app

def init_sample_data():
    """Initialize database with sample users if empty"""
    from models import User

    # Check if users already exist
    if User.query.first():
        print("✓ Database already initialized with sample data")
        return

    print("Initializing database with sample users...")

```

try:

Create sample users for each role

```
manufacturer = User(  
    email='manufacturer@ayurveda.com',  
    full_name='Rajesh Kumar',  
    role='Manufacturer',  
    company_name='Herbal Labs India',  
    contact_number='+91-9876543210',  
    address='Haridwar, Uttarakhand'  
)  
manufacturer.set_password('123456')
```

```
wholesaler = User(  
    email='wholesaler@ayurveda.com',  
    full_name='Amit Sharma',  
    role='Wholesaler',  
    company_name='Green Distributors Pvt Ltd',  
    contact_number='+91-9876543211',  
    address='Delhi, India'  
)  
wholesaler.set_password('123456')
```

```
retailer = User(  
    email='retailer@ayurveda.com',  
    full_name='Priya Patel',  
    role='Retailer',  
    company_name='Wellness Store',  
    contact_number='+91-9876543212',  
    address='Mumbai, Maharashtra'  
)  
retailer.set_password('123456')
```

```
customer = User(  
    email='customer@ayurveda.com',  
    full_name='Rajesh Kumar',  
    role='Customer',
```

```

        contact_number='+91-9876543213',
        address='Mumbai, Maharashtra'
    )
    customer.set_password('123456')

db.session.add_all([manufacturer, wholesaler, retailer, customer])
db.session.commit()

print("✓ Sample users created successfully!")
print("\n" + "="*60)
print("LOGIN CREDENTIALS (for testing):")
print("="*60)
print("Manufacturer: manufacturer@ayurveda.com / 123456")
print("Wholesaler: wholesaler@ayurveda.com / 123456")
print("Retailer: retailer@ayurveda.com / 123456")
print("Customer: customer@ayurveda.com / 123456")
print("="*60 + "\n")

except Exception as e:
    db.session.rollback()
    print(f"Error creating sample data: {e}")

if __name__ == '__main__':
    app = create_app()

    # Ensure instance folder exists for SQLite database
    os.makedirs('instance', exist_ok=True)

    print("\n" + "="*60)
    print("🌿 AYURVEDIC TRACE - QR-BASED SUPPLY CHAIN SYSTEM")
    print("="*60)
    print("Server starting on http://localhost:5000")
    print("API endpoints available at http://localhost:5000/api")
    print("="*60 + "\n")

    app.run(debug=True, port=5000)

```

III. INDIVIDUAL CONTRIBUTION

MEMBER 1: NISHAA S (7376232IT509)

- Developed backend APIs using Node.js/Python for handling product data and supply chain transactions.
- Designed and implemented the database (MySQL/MongoDB) for storing product details, batch information, and transaction records.
- Integrated blockchain technology (Ethereum/Hyperledger) to store immutable product records securely.
- Created and deployed smart contracts for automating supply chain processes and validations.
- Implemented QR code generation and linking with blockchain records for product authentication.
- Ensured secure data flow between frontend, backend, and blockchain network.
- Tested system performance, fixed bugs, and optimized transaction efficiency.

MEMBER 2: PERIYANAYAGI S (7376222IT221)

- Designed and developed user-friendly frontend interfaces using HTML, CSS, and JavaScript.
- Created responsive web pages for manufacturers, distributors, retailers, and consumers.
- Implemented QR code scanning functionality for real-time product verification.
- Developed dashboards to display product details, transaction history, and supply chain tracking.
- Integrated frontend with backend APIs and blockchain for seamless data visualization.
- Improved UI/UX design for better user interaction and accessibility.

- Conducted testing and debugging to ensure smooth system performance across devices.

IV. PUBLICATION PROOF

NISHAA S (7376232IT509)



PERIYANAYAGI S (7376222IT221)

